

Design and Analysis of Software Systems

Assignment 2 Part 2 : Integration Testing

Meet Parekh (2024101122)

1 System Design Overview

The **StreetRace Manager** is a command-line system for managing underground street races, crew members, vehicles, and missions. The system is built using a modular architecture where each module handles specific functionality and communicates with other modules through well-defined interfaces. A central **SystemState** object maintains all shared state including crew members, inventory, races, missions, and financial ledger.

1.1 Core Modules

- **Registration:** Registers crew members with validated attributes such as name, age, experience and skills.
- **Crew Management:** Manages roles, validates drivers, and tracks member availability status.
- **Inventory:** Maintains cars, parts, tools and tracks car condition and status.
- **Race Management:** Creates races, validates driver and car selection, and controls race lifecycle.
- **Results:** Computes race outcomes, updates ratings, distributes prizes and applies damage.
- **Mission Planning:** Assigns missions, validates required roles, and manages mission lifecycle.

1.2 Additional Modules

Finance Module: The Finance module is responsible for maintaining a comprehensive ledger of all income and expenses and updating the crew's cash balance atomically. It acts as a central integration layer between modules such as inventory, results, and missions. For example, race prizes and mission bounties are recorded as income entries, while car purchases, part/tool acquisitions, and repair costs are recorded as expenses. All cash mutations flow through `record_income()` and `record_expense()` functions, ensuring a single source of truth for financial data. This module was chosen because it enables traceability of all transactions and enforces the business rule that cash should never go negative, making the system auditable and preventing invalid states.

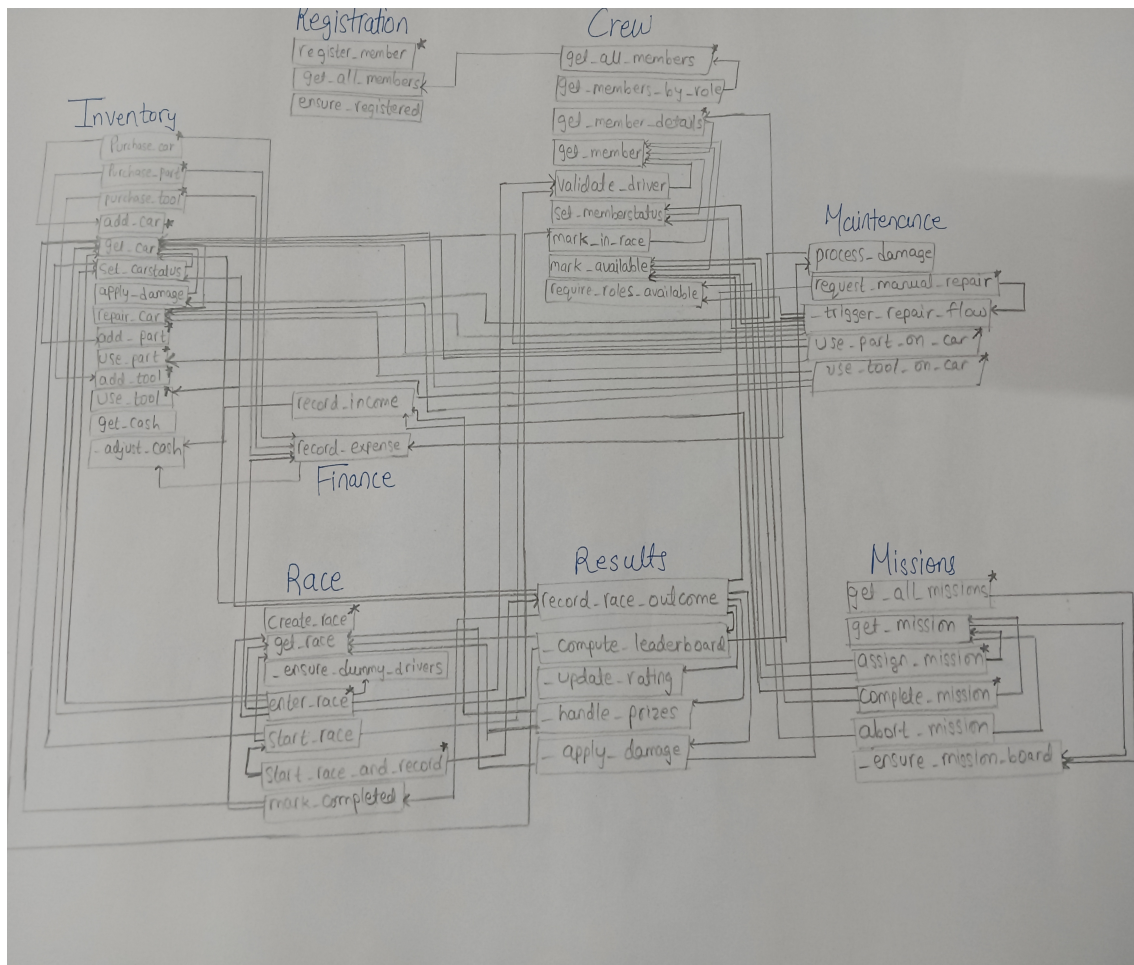
Maintenance Module: The Maintenance module handles car damage processing, repair workflows, and usage of parts/tools for repairs. It serves as a critical integration point between crew (for mechanic availability checks), inventory (for car condition updates), and finance (for repair cost deductions). When a car is damaged after a race, this module applies the damage through inventory and can trigger repair flows that require an available mechanic. Parts and tools can be used to incrementally repair cars, with each part restoring 5 condition points. This module was chosen to simulate real-world vehicle lifecycle management and create deeper cross-module interactions that test atomicity (e.g., parts should not be consumed if the car doesn't exist) and state restoration on failure.

2 Business Rules

The following business rules govern module interactions:

- A crew member must be **registered** before any role or skill actions apply.
- Only crew members with role **driver** may be entered in a race.
- A crew member can be in only one activity at a time (**Available**, **In Race**, or **In Mission**).
- Car condition must always be between 0–100; car is marked **Damaged** only when condition ≤ 0 .
- Experience must be **less than** age for any crew member.
- Missions require specific roles; assignment fails if required roles are unavailable.
- All cash changes must flow through the Finance module via `record_income()` or `record_expense()`.
- Purchases must be rejected if they would make cash negative (atomicity).
- Parts/tools must not be consumed if the target car doesn't exist (atomicity).
- Parts/tools should not be used on cars already at 100% condition.
- Manual repair requires an available mechanic; mechanic status must be restored on failure.
- Race damage depends on track difficulty, driver skill, and finish position.
- Prize money is only awarded to real crew members, not dummy drivers.
- Repair effect should scale with the quantity of parts used.

3 Task 2.1: Call Graph



How the call graph was constructed: The graph was drawn by first identifying all top-level functions defined in each module using the source files, and then tracing every direct function call made within and between modules. Each node represents a single function. Arrows point from caller to callee, following the direction of the call. All six core modules : **registration**, **crew**, **inventory**, **race**, **results**, and **mission** are shown in full, meaning every function they define appears as a node and every call they make (both intra-module and cross-module) is represented by an arrow. The two additional modules, **finance** and **maintenance**, are also included because they form critical integration hubs: **finance** is called by inventory, race, results, and mission to centralise all cash mutations, while **maintenance** bridges results, crew, inventory, and finance during damage and repair workflows. Nodes marked with a **star (★)** denote functions that are also invoked by the **ui** module, these are the integration entry points exposed to the command-line interface.

4 Task 2.2: Integration Test Design

A total of **57 test cases** were designed to validate module interactions. Each test case targets specific integration chains and business rules. Below is the complete list with scenarios and explanations of why each test is needed.

4.1 Registration → Crew Integration

Test Scenario	Why This Test Is Needed
Register driver and enter race	Verifies the happy path: a registered driver can successfully enter a race with entry fee deducted.
Enter race without registration	Ensures unregistered members cannot race; tests crew validation integration.
Enter race with non-driver role	Ensures only drivers can race; tests role validation.
Registration creates crew member	Tests registration-crew integration.
Get members by role	Tests crew filtering functionality.
Age-experience validation when age missing	Ensures experience < age rule is enforced even when age defaults to 0.

4.2 Race → Results → Finance Integration

Test Scenario	Why This Test Is Needed
Complete race with winner	Verifies prize distribution, rating updates, cash increase, and status reset after race.
Race leaderboard computation	Ensures leaderboard is correctly computed with all participants.
Race with damage application	Verifies car condition is reduced after race and status updated appropriately.
High difficulty causes more damage	Tests that track difficulty parameter affects damage calculation.
Race with multiple dummy drivers	Ensures leaderboard includes all participants (real and dummy drivers).
Ratings update after race	Verifies driver rating increases based on finish position.(only top 2)
Race entry and prize ledger entries	Ensures both expense (entry fee) and income (prize) are recorded in ledger.

4.3 Mission → Crew → Finance Integration

Test Scenario	Why This Test Is Needed
Assign and complete mission	Tests full mission lifecycle: assignment, crew busy status, bounty on completion.
Assign mission without required roles	Ensures mission fails if required crew roles are unavailable.

Test Scenario	Why This Test Is Needed
Mission abort	Verifies crew is released and no bounty paid on abort.
Mission with multiple crew roles	Tests that missions requiring multiple roles assign all correctly.
Complete mission twice	Ensures duplicate completion is rejected.
Double role mission assignment	Tests that same person isn't assigned twice when mission needs 2 drivers.
Mission completion without bounty	Verifies free missions (bounty=0) complete without ledger entry.

4.4 Maintenance → Crew → Inventory → Finance Integration

Test Scenario	Why This Test Is Needed
Manual repair with mechanic	Tests complete repair flow: mechanic assigned, cost deducted, car repaired.
Manual repair without mechanic	Ensures repair fails if no mechanic is available.
Use part for repair	Verifies part is consumed and car condition improves.
Use tool for repair	Verifies tool is consumed and condition improves.
Repair car with max condition	Tests boundary: car at 100% should reject repair.
Use nonexistent part	Ensures error when trying to use part not in inventory.
Use part with insufficient quantity	Tests quantity validation before use.
Damage and repair workflow	Tests complete cycle: race damage → repair → car usable again.
Parts and tools inventory management	Verifies add/use operations track quantities correctly.
Part consumption requires valid car	Ensures parts are NOT consumed if car doesn't exist (atomicity).
Tool consumption requires valid car	Ensures tools are NOT consumed if car doesn't exist (atomicity).
Mechanic status restored on failed repair	Verifies mechanic returns to Available if repair fails due to insufficient cash.
Bulk parts repair scales with quantity	Tests that using 5 parts repairs 25 condition (5 per part), not just 5.
No part consumption at max condition	Ensures parts aren't wasted on cars already at 100% condition.

4.5 Inventory → Finance Integration (Purchases)

Test Scenario	Why This Test Is Needed
Purchase car	Verifies car added, cash deducted, ledger updated.
Purchase part	Tests part purchase updates inventory and finance.
Purchase tool	Tests tool purchase updates inventory and finance.
Purchase with insufficient cash	Ensures purchase rejected when cash is too low.

Test Scenario	Why This Test Is Needed
Insufficient funds prevents purchase atomicity	Tests that failed purchase leaves state unchanged.

4.6 Status Constraint Enforcement

Test Scenario	Why This Test Is Needed
Driver cannot enter race while in mission	Critical test: ensures mutual exclusion of activities.
Driver cannot enter multiple races	Prevents same driver in concurrent races.
Car cannot be used while in race	Prevents same car in concurrent races.
Damaged car cannot enter race	Ensures damaged cars are blocked from racing.
Mechanic cannot enter another mission	Tests mission overlap prevention for crew.
Crew status transitions	Verifies status changes correctly through activities.
Car status lifecycle	Tests car status state machine (Available $\rightarrow$ In Race $\rightarrow$ Damaged).

4.7 End-to-End Workflow Tests

Test Scenario	Why This Test Is Needed
Full race lifecycle	Tests registration $\rightarrow$ race $\rightarrow$ results $\rightarrow$ finance end-to-end.
Crew member workflow across race and mission	Tests sequential transitions: race then mission.
Multiple races sequential	Verifies driver can race again after completing previous race.

4.8 Edge Cases and Error Handling

Test Scenario	Why This Test Is Needed
Race with nonexistent car	Tests error handling for missing entities.
Start race without driver	Tests validation for race start conditions.
Insufficient cash for race entry	Tests cash validation before race entry.
Damaged car status after mark completed	Ensures car remains Damaged if condition ≤ 0 after race.
Direct mark completed respects damage	Tests that damage status is preserved.
Cash non-negative on expensive repairs	Ensures system rejects transactions making cash negative.

4.9 Financial Integrity Tests

Test Scenario	Why This Test Is Needed
Ledger records all transactions	Verifies all cash changes are recorded in ledger.
Cash balance matches ledger	Tests that cash = initial + income - expenses.

5 Errors Detected and Fixed

During integration testing, **6 errors** were detected and fixed. All identified issues were fixed.

Error	Issue	Fix Applied
1	Part consumed even when car doesn't exist	Added car existence validation in <code>use_part_on_car()</code> before consuming part.
2	Tool consumed even when car doesn't exist	Added car existence validation in <code>use_tool_on_car()</code> before consuming tool.
3	Experience $\geq$ age allowed when age=0	Added validation to enforce age requirement when experience is provided.
4	Mechanic stuck in "In Mission" after failed repair	Wrapped repair logic in try-finally to restore mechanic status on failure.
5	Using 5 parts only repaired 5 condition	Changed repair calculation to scale with quantity: <code>5 * quantity</code> .
6	Part consumed on car with 100% condition	Added condition check to prevent repair on perfect cars.